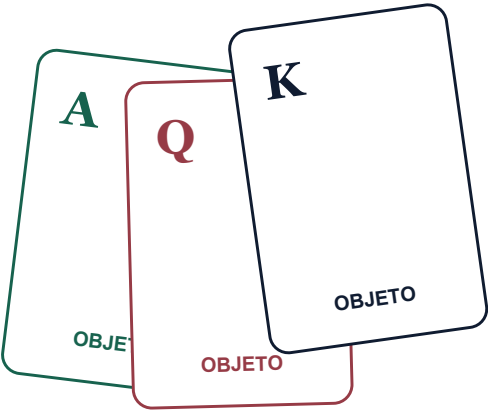


Blackjack em Camadas

Roadmap guiado para concluir o projeto atual sem apagar o trabalho intelectual do estudante.

Diagnóstico factual, contratos, exercícios, testes e critérios de pronto.

Snapshot do projeto: 24 de agosto de 2026



ANTES DE COMEÇAR

O propósito deste roteiro

Este documento transforma o estado atual do seu projeto em um caminho de estudo. Ele não substitui seu raciocínio com classes prontas: mostra fatos, dependências, contratos, perguntas e testes que permitem construir cada parte conscientemente.

RESULTADO DA AUDITORIA

O projeto compila e os três testes atuais passam. Entretanto, ele ainda não executa uma rodada: mostra a mensagem de preparação, cria os objetos e retorna ao menu. A prioridade imediata é fazer a mão conservar cartas.

Como este PDF deve ser usado

1. Leia uma fase inteira antes de alterar o código.
2. Escreva os testes-alvo ou, no mínimo, descreva Dado/Quando/Então.
3. Implemente o menor comportamento que satisfaz o contrato.
4. Execute `mvn clean test`.
5. Explique em voz alta por que o método pertence àquela classe.
6. Atualize o PUMML apenas depois que o código representar a relação.

O que este material não fará

- Não implementará apostas, split, double down, seguro ou múltiplos jogadores no primeiro ciclo.
- Não recomendará getters e setters automáticos para todos os campos.
- Não tratará o terminal como lugar das regras do domínio.
- Não exigirá conceitos que ainda não melhoram o projeto, como herança prematura.
- Não confundirá código que compila com comportamento correto.

REGRA DE ESTUDO

Cada mudança precisa responder a três perguntas: qual problema ela resolve, qual objeto possui os dados necessários e qual teste prova o comportamento.

LEGENDA

Como ler diagnósticos e exercícios

Rótulo	Significado	Sua ação
ESTÁVEL E TESTADO	Existe, representa a intenção atual e possui ao menos uma verificação útil.	Preserve; mude somente com teste.
PARCIAL	Há uma base correta, mas faltam invariantes ou comportamento.	Complete em incrementos pequenos.
AUSENTE	A responsabilidade ainda não foi implementada.	Defina contrato antes do corpo.
DIVERGENTE	Código, PUML ou documentação contam histórias diferentes.	Escolha a fonte de verdade e sincronize.

Símbolos editoriais

CONCEITO CENTRAL

Explica a ideia que deve sobreviver ao exercício.

PARE E PENSE

Interrompa a leitura e formule sua própria resposta.

ARMADILHA

Mostra um caminho que compila, mas enfraquece o modelo.

DIAGNÓSTICO

Relaciona um sintoma atual com sua causa concreta.

NAVEGAÇÃO

Sumário

O propósito deste roteiro	2
Como este PDF deve ser usado	2
O que este material não fará	2
Como ler diagnósticos e exercícios	3
Símbolos editoriais	3
Snapshot técnico do projeto	8
Execução observada	8
Leitura correta desse resultado	8
Matriz de saúde por componente	9
Grafo de objetos atual	10
Débito de inicialização em Blackjack	10
Escopo do Blackjack mínimo	11
MVP recomendado	11
Explicitamente fora do primeiro ciclo	11
Vocabulário de regras	12
Tabela de decisões do projeto	12
Mapa de dependências	13
Ordem recomendada	13
Por que não começar pelo menu?	14
Ler o sistema antes de alterá-lo	15
Fazer a mão e os participantes conservarem estado	18
Fechar o domínio de Carta	21
Completar o ciclo de vida do Baralho	24

A transferência de uma carta	27
Invariante de conservação	27
Experimento de referência	27
Transformar Blackjack em coordenador	28
Calcular pontuação no lugar correto	31
Teste manual de múltiplos ases	34
Ficha de previsão	34
Modelar turnos e resultado da rodada	35
Máquina de estados mínima	38
Tabela de transições para completar	38
Conectar o domínio à interface	39
Usar testes, UML e README como especificação	42
Matriz completa de testes	45
Modelo Dado / Quando / Então	46
Arrange, Act, Assert	46
Aleatoriedade sem testes frágeis	46
Testes que parecem bons, mas não bastam	47
Ordem de execução recomendada	47
Laboratório de UML	48
Problemas atuais de blackjack.puml	48
Modelo-alvo conceitual	49
Relações e multiplicidades	50
Esqueleto PlantUML para você completar	50
Checklist de sincronização do diagrama	51
Perguntas de revisão	51

Contratos do modelo-alvo	52
Invariantes por classe	53
Sinais de encapsulamento saudável	53
Definição de pronto do MVP	54
Domínio e estado	54
Fluxo da partida	54
Qualidade técnica	55
Documentação e modelo	55
Roteiro de aceitação manual	56
Registro de evidência	56
Plano de estudo em doze sessões	57
Ritual de cada sessão	57
Ficha reutilizável de implementação	58
Pistas graduais	59
1. Mao não conserva cartas	59
2. Jogador perde a mão	59
3. Carta aceita XPTO	60
4. Comprar não reduz o baralho	60
5. Distribuição cria cartas novas	61
6. A + A soma 22	61
7. Teste de shuffle falha às vezes	62
8. Nome completo quebra o menu	62
Glossário aplicado	63
Sua próxima ação concreta	65
Sequência das próximas três entregas	65

NOTAS DA EDIÇÃO

Como usar este roteiro impresso

Esta página reúne os sinais de progresso, a rotina de estudo e o caminho até o encerramento.

ANTES DE PROGRAMAR

1. Leia a fase inteira.

2. Preveja o comportamento.

3. Escreva o teste-alvo ou descreva sua evidência.

AO CONCLUIR UMA FASE

1. Execute todos os testes.

2. Explique a responsabilidade em voz alta.

3. Atualize UML e README somente após o código.

Legenda de progresso

ESTÁVEL E TESTADO

preserve; mude somente com teste

PARCIAL

complete em incrementos pequenos

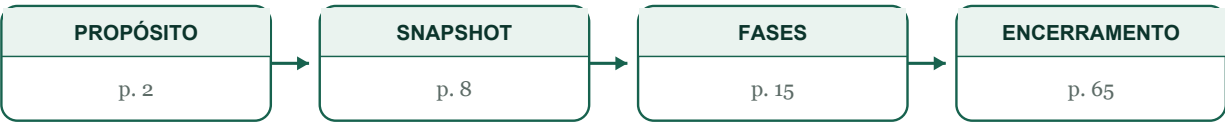
AUSENTE

defina o contrato antes do corpo

DIVERGENTE

escolha a fonte de verdade e sincronize

Rota rápida



REGISTRO DE ESTUDO

Marque no papel o que você previu, o que o teste demonstrou e qual decisão de projeto mudou. O roteiro termina com as Perguntas finais de professor, na página 65.

CAPÍTULO 01

01

Snapshot técnico do projeto

Uma fotografia factual do que existe em 24 de agosto de 2026, antes que novas mudanças alterem o diagnóstico.

BASELINE CONFIRMADO

`mvn clean test` recompila 8 fontes, executa 3 testes e termina com **BUILD SUCCESS**. Isso prova que a base compila e que os três testes passam; não prova que o Blackjack esteja jogável.

Execução observada

TRAÇO DE EXECUÇÃO

- 1) O menu é exibido.
- 2) A opção 1 solicita um nome.
- 3) A mensagem "Estamos preparando seu jogo..." aparece.
- 4) App cria Jogador, Baralho, Dealer e Blackjack.
- 5) Nenhuma carta é distribuída.
- 6) O programa volta ao menu.

Leitura correta desse resultado

- A infraestrutura Maven e JUnit está funcionando.
- Naipes, Carta e a montagem inicial do Baralho já têm uma base útil.
- O sistema ainda não possui fluxo de partida; instanciar Blackjack não executa automaticamente uma rodada.
- O objeto local chamado jogo é criado em App e não recebe nenhuma mensagem depois disso.

ERRO DE INTERPRETAÇÃO

Objeto construído não significa comportamento executado. Um construtor estabelece um estado inicial; uma operação explícita deve iniciar ou conduzir a rodada.

Matriz de saúde por componente

Componente	Estado	Base existente	Próximo avanço
Naípe	ESTÁVEL E TESTADO	Enum com quatro valores e símbolo.	Aparecer no PUML; ampliar testes apenas quando útil.
Carta	PARCIAL	Campos finais, validação básica, normalização e getters.	Impedir ranks inválidos; definir valor-base, texto e igualdade.
Baralho	PARCIAL	Monta 52 cartas e informa quantidade.	Embaralhar, comprar, vazio, conteúdo verificável e testes de unicidade.
Mao	AUSENTE	A classe existe, mas a lista é local do construtor.	Criar campo persistente e operações de domínio.
Jogador	PARCIAL	Conserva nome.	Encapsular nome e conservar uma mão própria.
Dealer	PARCIAL	Conserva uma Mao como campo.	Encapsular, tornar final e definir política de compra.
Blackjack	AUSENTE	Recebe dependências no construtor.	Remover objetos redundantes e coordenar distribuição, turnos e resultado.
App	PARCIAL	Menu e criação inicial funcionam.	Entrada robusta, usar o jogo e exibir uma rodada.
Testes	PARCIAL	Três testes reais passam.	Separar por classe e cobrir invariantes e regras.
PUML	DIVERGENTE	Esboça as relações principais.	Corrigir sintaxe e sincronizar com código atual/alvo.
README	PARCIAL	Comandos Maven estão corretos.	Documentar escopo, regras, arquitetura e estado.

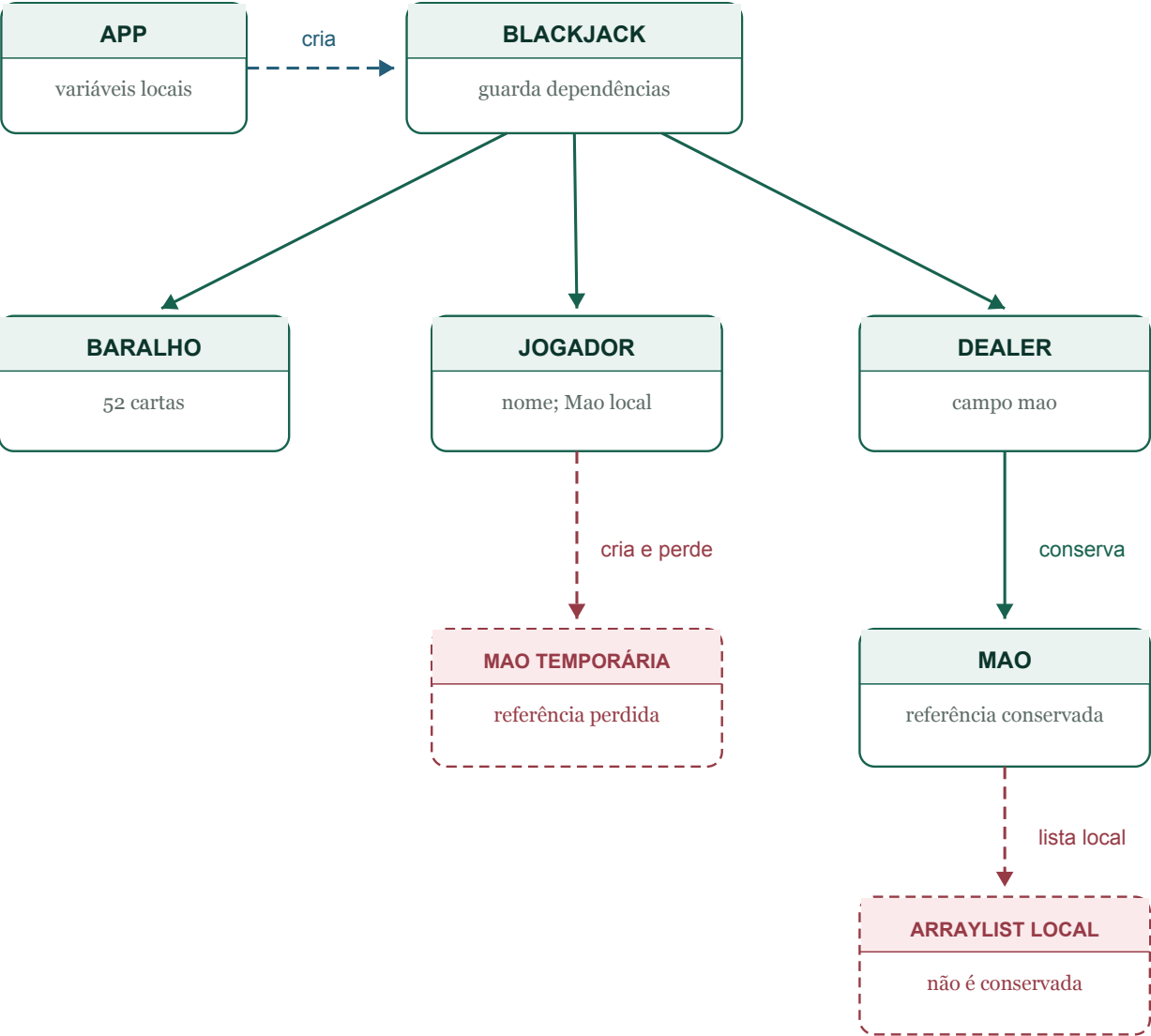
PRIORIDADE REAL

Mao é o bloqueador imediato. Enquanto sua lista existir apenas como variável local, não há lugar persistente para receber uma carta retirada do baralho.

MODELO VISUAL

Grafo de objetos atual

Snapshot: após os construtores e antes de playBlackjack retornar. Linhas sólidas mostram referências conservadas; linhas tracejadas mostram dependências ou objetos temporários.



Tracejada azul: dependência temporária

Sólida verde: referência conservada

Tracejada vermelha: estado perdido

CONSERVADO

Blackjack guarda Baralho, Jogador e Dealer. Dealer guarda sua Mao.

PERDIDO

Jogador cria uma Mao local. Cada Mao cria uma lista local e não a conserva.

REDUNDANTE

Blackjack cria Baralho, Jogador e Dealer temporários antes de substituir os campos.

CAPÍTULO 02

02

Escopo do Blackjack mínimo

Antes dos métodos, feche as regras. Este MVP usa uma variante simples, explícita e testável.

MVP fechado

- ☐ Um jogador humano contra a banca (classe Dealer).
- ☐ Um baralho padrão de 52 cartas, sem curingas e sem descarte durante a rodada.
- ☐ Duas cartas para cada participante, distribuídas alternadamente.
- ☐ A segunda carta da banca fica oculta somente na apresentação; ela permanece na Mão do Dealer.
- ☐ Os dois blackjacks naturais são avaliados logo após a distribuição.
- ☐ O jogador pode pedir ou parar; ao atingir 21 ou estourar, seu turno termina.
- ☐ A banca compra abaixo de 17 e para em 17 ou mais, inclusive soft 17 (regra S17).
- ☐ Resultado é VITORIA_JOGADOR, VITORIA_BANCA ou EMPATE; o motivo é registrado separadamente.
- ☐ Nova rodada cria novo Baralho, novo Dealer e novas mãos, preservando apenas o nome do jogador.

Fora do primeiro ciclo

- Apostas, fichas, pagamentos e saldo.
- Split, double down, seguro e surrender.
- Múltiplos jogadores, vários baralhos e carta de corte.
- Persistência, interface gráfica e regras de cassino adicionais.

REGRA DE APRESENTAÇÃO

Ocultar uma carta não muda sua posse. O Dealer conserva as duas cartas; a interface apenas recebe uma visão adequada ao estado da rodada.

Precedência, vocabulário e decisões fechadas

A ordem abaixo impede resultados contraditórios e deve aparecer no README e nos testes.

Ordem de resolução

- 1. Após distribuir, se ambos tiverem natural: EMPATE / AMBOS_NATURAIS.
- 2. Se apenas o jogador tiver natural: VITORIA_JOGADOR / BLACKJACK_NATURAL.
- 3. Se apenas a banca tiver natural: VITORIA_BANCA / BLACKJACK_NATURAL.
- 4. Sem naturais, o jogador age; 21 encerra o turno e estouro encerra a rodada.
- 5. Se o jogador não estourou, revele a carta oculta e execute a banca pela regra S17.
- 6. Estouro da banca dá a vitória ao jogador; caso contrário, compare os totais.
- 7. Maior total vence; totais iguais resultam em EMPATE. Natural sempre precede 21 comum.

RESULTADO NÃO É MOTIVO

Resultado: VITORIA_JOGADOR, VITORIA_BANCA ou EMPATE. MotivoResultado: BLACKJACK_NATURAL, ESTOURO, MAIOR_TOTAL, TOTAIS_IGUAIS ou AMBOS_NATURAIS. Assim, 'estouro' nunca aparece sozinho como vencedor.

Vocabulário adotado

Termo	Definição operacional	Efeito
Natural	21 com duas cartas: ás + valor-base 10.	Avaliado antes dos turnos.
21 comum	Total 21 com três ou mais cartas.	Encerra turno, mas não é natural.
Mão macia	Ao menos um ás ainda conta como 11.	Permite distinguir soft 17.
Soft 17	17 com um ás contado como 11.	A banca para neste MVP.
Carta oculta	Carta presente na Mao, omitida da exibição.	Revelada antes do turno da banca.

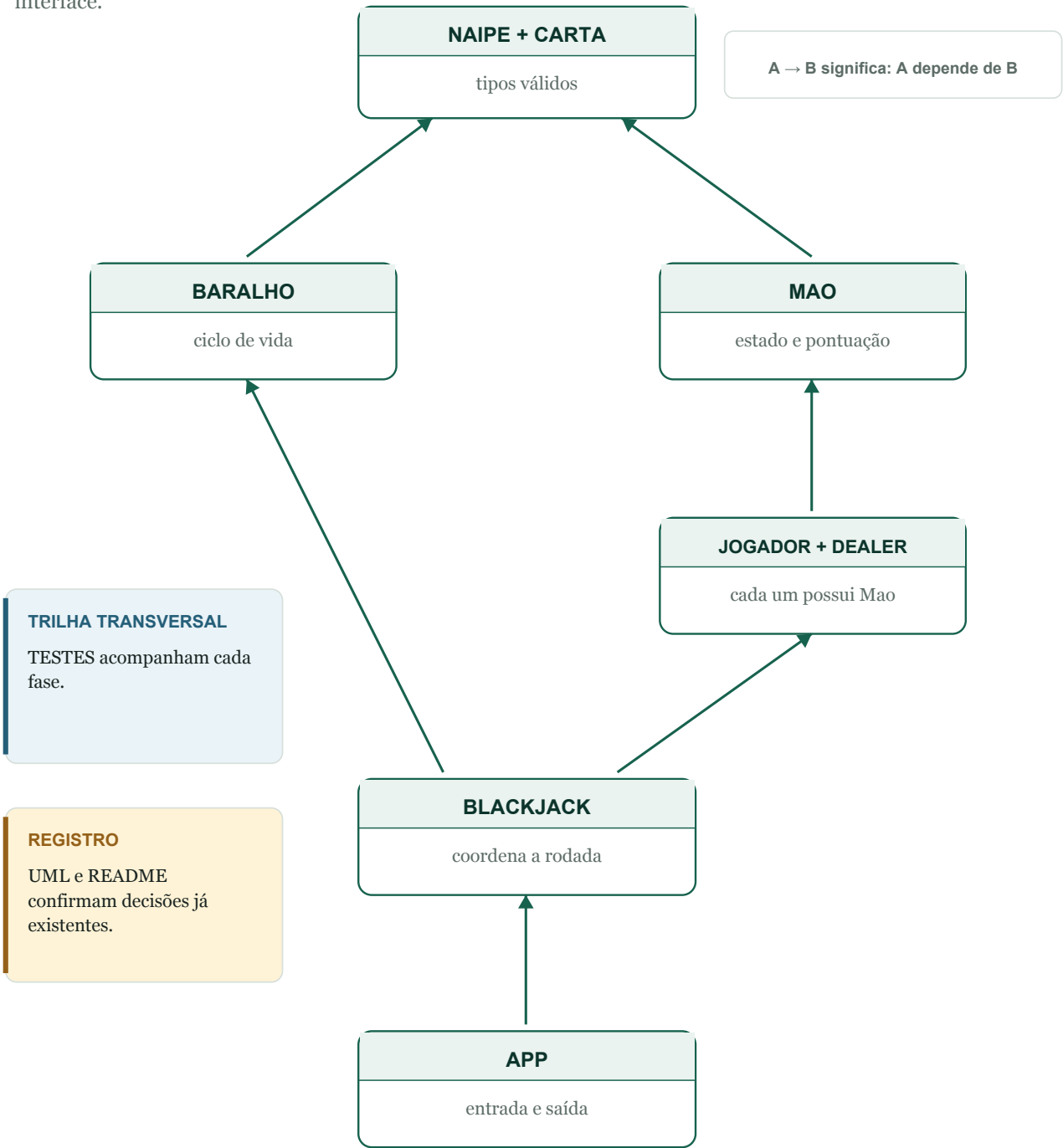
NOVA RODADA

O MVP não limpa objetos antigos: App monta um novo jogo com novo baralho e novas mãos. Apenas o nome validado pode ser reutilizado.

CAPÍTULO 03

Mapa de dependências

Implemente primeiro o estado que recebe as cartas; depois permita o movimento; por fim coordene regras e interface.



Ordem segura

- 1. Persistir cartas em Mao.
- 2. Dar uma Mao própria a cada participante.
- 3. Fechar o domínio de Carta.
- 4. Comprar e embaralhar no Baralho.
- 5. Distribuir pelo Blackjack.
- 6. Calcular pontuação e turnos.
- 7. Conectar a interface e sincronizar documentos.

Por que não começar pelo menu?

O menu consegue perguntar 'pedir ou parar', mas ainda não existe uma mão que guarde a carta, um baralho que a retire, nem um jogo que coordene a transferência. Começar pela interface obrigaria App a assumir regras que depois precisariam ser movidas.

REGRA DE DEPENDÊNCIA

Uma camada deve conversar com comportamentos estáveis da camada anterior. App depende de Blackjack; Blackjack depende de Baralho, participantes e mãos; mãos e baralho dependem de Carta.

CAPÍTULO F0

F0

Ler o sistema antes de alterá-lo

Depois que um construtor termina, quem ainda aponta para cada objeto criado?

OBJETIVO DA FASE

Construir um modelo mental preciso de classe, objeto, referência, campo e variável local antes de adicionar regras.

O que o código atual revela

- `Mao` cria `ArrayList<Carta> mao` dentro do construtor; essa variável não é um campo.
- Jogador cria `Mao mao` dentro do construtor e não conserva a referência.
- Dealer declara `Mao mao` diretamente no corpo da classe; portanto conserva a referência.
- Blackjack mistura dependências recebidas com objetos construídos antecipadamente.

Conceitos que você está praticando

- Classe versus instância.
- Variável de referência versus objeto referenciado.
- Escopo léxico e alcançabilidade.
- Campos como estado persistente.
- Construtor como estabelecimento de invariantes.
- Coleta de lixo sem a simplificação 'apagado imediatamente'.

PARE E PENSE

Depois que um construtor termina, quem ainda aponta para cada objeto criado?

Decisões antes do código

1. Marque cada declaração atual como campo, parâmetro ou variável local.
2. Desenhe o grafo de referências após `new Blackjack(...)`.
3. Liste quais objetos são obrigatórios para um jogo válido.
4. Decida quais construtores criam dependências e quais as recebem.

Missão prática

- ☐ Anotar, no próprio caderno, o escopo de cada variável relevante.
- ☐ Prever a saída e o grafo antes de executar o programa.
- ☐ Identificar todos os campos `package-private` que deveriam proteger estado.
- ☐ Explicar por que `final List<Carta>` não torna a lista imutável.
- ☐ Registrar a baseline: 3 testes passando.

Contratos sugeridos

Operação	Responsabilidade	Não deve fazer
Construtor	Deixar o objeto válido ao terminar.	Executar a partida inteira ou esconder efeitos inesperados.
Campo	Conservar estado necessário depois do método.	Virar público apenas para facilitar um teste.
Variável local	Apoiar um cálculo temporário.	Representar estado que o objeto precisa lembrar.

Testes-alvo

Cenário	Evidência observável
Mao recém-criada	A API pública deve revelar quantidade zero quando for implementada.
Duas referências para a mesma Mao	Alterações feitas pela primeira referência são observadas pela segunda.
Duas instâncias de Mao	Alterar uma não altera a outra.

Armadilhas previsíveis

- Dizer que a variável contém o objeto; ela contém uma referência que permite alcançá-lo.
- Dizer que sair do escopo destrói o objeto imediatamente.
- Transformar campos em `static` para 'não perder' o valor.
- Criar getters públicos para toda coleção apenas para inspecioná-la.

Pronto quando

- ☐ Você consegue classificar campos, parâmetros e locais sem executar o código.
- ☐ Você explica por que a lista de Mao se torna inalcançável.
- ☐ Você identifica os objetos redundantes de Blackjack.
- ☐ Nenhuma regra do jogo foi adicionada antes de entender o estado.

Explique em voz alta

- Qual é a diferença entre construir e armazenar?
- Por que Dealer conserva Mao e Jogador não?
- O que `this` seleciona?
- Por que Java passa uma cópia do valor da referência?

CAPÍTULO F1

F1

Fazer a mão e os participantes conservarem estado

Onde a coleção deve viver para continuar existindo depois do construtor?

OBJETIVO DA FASE

Criar o primeiro grafo de objetos estável: cada participante possui uma mão própria, e cada mão conserva suas cartas.

O que o código atual revela

- Mao termina o construtor sem qualquer campo.
- Jogador conserva o nome, mas a Mao criada é apenas local.
- Dealer conserva uma Mao, porém o campo não é privado nem final.
- Os imports de ArrayList em Jogador e Dealer não são usados.

Conceitos que você está praticando

- Associação de posse; no MVP, pode ser composição se a mão tiver ciclo de vida exclusivo do participante.
- Estado por instância; perigo de compartilhar com `static`.
- Programar para a interface `List<Carta>`.
- Encapsulamento por comportamento, não por getters/setters automáticos.
- Aliasing e cópia defensiva de coleções.

PARE E PENSE

Onde a coleção deve viver para continuar existindo depois do construtor?

Decisões antes do código

1. Escolha o nome do campo da coleção em Mao: evite chamar a coleção e a classe pelo mesmo conceito sem clareza.
2. Decida quais operações mínimas provam que a mão conserva estado.
3. Decida se Jogador e Dealer criam a própria Mao ou a recebem no construtor.
4. Defina a invariante do nome do jogador.
5. Escolha como inspecionar cartas sem devolver a lista mutável original.

Missão prática

- ☐ Criar em Mao um campo privado e final do tipo `List<Carta>`.
- ☐ Inicializar a coleção no construtor.
- ☐ Criar uma operação para receber uma carta não nula.
- ☐ Criar uma consulta de quantidade.
- ☐ Fazer Jogador conservar sua Mao em campo.
- ☐ Encapsular e validar o nome.
- ☐ Tornar a Mao de Dealer privada e final.
- ☐ Remover imports e construtores vazios que não agregam comportamento.

Contratos sugeridos

Operação	Responsabilidade	Não deve fazer
Mao.adicionar(Carta)	Conservar uma carta válida na coleção interna.	Buscar a carta no baralho ou imprimir mensagens.
Mao.getQuantidadeCartas()	Informar o tamanho atual.	Expor a implementação concreta da coleção.
Jogador.getMao()	Permitir colaboração controlada com a mão.	Trocar a mão por meio de setter.
Jogador.getNome()	Informar o nome validado.	Alterar o nome como efeito colateral.
Dealer.getMao()	Dar acesso ao comportamento da própria mão.	Compartilhar uma mão global.

Testes-alvo

Cenário	Evidência observável
Nova Mao	Quantidade igual a zero.
Adicionar uma carta	Quantidade aumenta para um e a referência recebida é conservada.
Adicionar null	A mão rejeita segundo o contrato escolhido.
Duas mãos	Adicionar em uma não altera a outra.
Jogador e Dealer	Cada um possui uma instância diferente de Mao.
Nome vazio	Jogador inválido não nasce ou a política escolhida é explícita.

Armadilhas previsíveis

- Declarar a lista dentro do construtor novamente.
- Usar `static List<Carta>` e fazer todas as mãos compartilharem cartas.
- Devolver a lista interna mutável para App chamar `add` diretamente.
- Guardar quantidade e lista separadamente, permitindo divergência.
- Criar herança entre Jogador e Dealer antes de existir comportamento comum significativo.

Pronto quando

- ☐ Uma carta adicionada ainda pode ser consultada depois que o método termina.
- ☐ Cada participante conserva exatamente sua própria mão.
- ☐ Campos de estado estão privados e referências obrigatórias estão finais.
- ☐ Os testes usam API pública e não reflexão para alcançar campos.

Explique em voz alta

- Por que a lista precisa ser campo, mas a variável temporária de um cálculo não?
- O que `final` impede em uma lista e o que não impede?
- Por que receber carta pertence à Mao?
- Que problema surgiria se duas mãos compartilhassem a mesma lista?

CAPÍTULO F2

F2

Fechar o domínio de Carta

Se naipe é um conjunto fechado, por que valor nominal ainda aceita qualquer texto?

OBJETIVO DA FASE

Impedir que cartas impossíveis nasçam e separar rank, valor-base e pontuação contextual.

O que o código atual revela

- Carta rejeita null e texto em branco, mas aceita XYZ, 14 ou -8.
- Naipe já demonstra como um enum substitui Strings livres.
- Baralho conhece os 13 ranks por meio de um array local.
- A identidade Q não deve ser substituída pela String "10" apenas porque sua pontuação-base é dez.

Conceitos que você está praticando

- Enum como tipo de domínio.
- Objeto de valor imutável.
- Identidade lógica versus identidade de referência.
- Valor-base versus valor derivado pelo contexto.
- Contrato de `equals` e `hashCode`.

PARE E PENSE

Se naipe é um conjunto fechado, por que valor nominal ainda aceita qualquer texto?

Decisões antes do código

1. Escolha um nome: `ValorCarta` ou `Rank`.
2. Defina as 13 constantes sem usar `ordinal()` como pontuação.
3. Decida se o enum conhece apenas o valor-base ou também perguntas como `ehAs()`.
4. Decida se duas cartas de mesmo rank e naipe são logicamente iguais neste modelo de um baralho.
5. Defina uma representação textual legível sem depender de símbolos incertos.

Missão prática

- ☐ Criar o enum dos 13 valores.
- ☐ Alterar `Carta` para receber o novo tipo em vez de `String`.
- ☐ Atualizar `Baralho` para percorrer os valores válidos.
- ☐ Manter `Carta` sem setters e com campos finais.
- ☐ Adicionar representação textual útil para terminal e testes.
- ☐ Se a decisão for objeto de valor, implementar igualdade e `hashCode` coerentes.
- ☐ Atualizar testes antes de remover todas as `Strings`.

Contratos sugeridos

Operação	Responsabilidade	Não deve fazer
<code>ValorCarta.getValorBase()</code>	Informar 2-10 e valor-base de figuras/ás.	Decidir sozinho se o ás vale 1 ou 11 na mão.
<code>Carta.getValorNominal()</code>	Informar o rank tipado da carta.	Converter Q em 10 e perder a identidade.
<code>Carta.toString()</code>	Produzir texto legível para interface e diagnóstico.	Imprimir diretamente no console como efeito colateral.
<code>Carta.equals(Object)</code>	Aplicar a igualdade lógica escolhida.	Usar campos diferentes dos usados por <code>hashCode</code> .

Testes-alvo

Cenário	Evidência observável
Todos os 13 ranks	Podem formar Carta com cada Naipes.
Rank inexistente	Não pode ser representado pelo tipo.
Figura	Continua sendo J, Q ou K e possui valor-base 10.
Ás isolado	Mantém identidade A; a mão decide sua interpretação.
Duas cartas equivalentes	<code>equals</code> e <code>hashCode</code> obedecem à política documentada.
Texto da carta	É legível e inclui rank e naipe.

Armadilhas previsíveis

- Usar `ordinal()` como valor; reordenar o enum mudaria a regra.
- Manter simultaneamente `String` e `enum` como duas fontes de verdade.
- Colocar a lógica completa de múltiplos ases dentro de `Carta`.
- Implementar `equals` sem `hashCode`.
- Dar setters a `Carta` para reaproveitar o mesmo objeto durante a montagem do baralho.

Pronto quando

- ☐ Nenhuma carta inválida pode nascer.
- ☐ Carta conserva rank e naipe durante toda a vida.
- ☐ Figuras não perdem sua identidade.
- ☐ Baralho não mantém uma lista paralela de `Strings` válidas.
- ☐ Os testes descrevem a política de igualdade.

Explique em voz alta

- Por que `Naipes.COPAS` é diferente da `String 'COPAS'`?
- Por que o ás exige contexto de `Mao`?
- O que significa duas cartas serem iguais neste projeto?
- Qual mudança futura quebraria um uso de `ordinal`?

CAPÍTULO F3

F3

Completar o ciclo de vida do Baralho

Que operação garante que comprar uma carta diminui exatamente uma unidade?

OBJETIVO DA FASE

Fazer o Baralho proteger suas cartas restantes, embaralhar a própria coleção e transferir uma carta por vez.

O que o código atual revela

- Baralho cria 52 cartas corretas e informa a quantidade.
- Ainda não existe operação para retirar uma carta.
- Blackjack possui um método estático e vazio chamado embaralhar, embora as cartas pertençam ao Baralho.
- O conteúdo interno não é exposto, o que preserva encapsulamento, mas exige APIs de domínio para testes e colaboração.

Conceitos que você está praticando

- Método de instância versus método estático.
- Mutação interna controlada.
- Remoção e transferência de referência.
- Política de coleção vazia.
- Aleatoriedade determinística em testes.

PARE E PENSE

Que operação garante que comprar uma carta diminui exatamente uma unidade?

Decisões antes do código

1. Escolha qual extremidade representa o topo.
2. Escolha entre lançar exceção clara ou devolver Optional quando vazio.
3. Decida se embaralhar recebe um Random para testes ou usa uma versão simples inicialmente.
4. Defina uma forma segura de verificar composição sem entregar a lista mutável.
5. Defina se um Baralho pode ser reiniciado ou se uma nova instância representa uma nova rodada.

Missão prática

- ☐ Mover a responsabilidade de embaralhar para Baralho.
- ☐ Criar uma operação que remova e devolva uma carta.
- ☐ Criar consulta para baralho vazio.
- ☐ Preservar a lista como detalhe privado.
- ☐ Testar as 52 combinações, não somente o tamanho.
- ☐ Testar esgotamento até a última carta.
- ☐ Documentar o comportamento da compra número 53.

Contratos sugeridos

Operação	Responsabilidade	Não deve fazer
Baralho.embaralhar()	Reordenar as cartas restantes da própria instância.	Criar outro baralho ou alterar cartas já entregues.
Baralho.comprar()	Remover e devolver uma única carta.	Devolver sem remover ou retornar null silenciosamente.
Baralho.estaVazio()	Responder a partir do tamanho atual.	Manter um boolean separado que possa ficar desatualizado.
Baralho.getQuantidadeCartas()	Informar quantas ainda estão disponíveis.	Expor a lista interna.

Testes-alvo

Cenário	Evidência observável
Baralho novo	52 cartas e 52 combinações únicas.
Distribuição por naipe	13 cartas de cada Naipes.
Distribuição por rank	Quatro cartas de cada ValorCarta.
Uma compra	Quantidade passa de 52 para 51.
Esgotamento	52 compras não repetem uma mesma carta.
Baralho vazio	O contrato escolhido é obedecido.
Embaralhamento	Quantidade e conjunto são preservados.

Armadilhas previsíveis

- Testar que embaralhar sempre muda a ordem; a mesma permutação é possível.
- Usar `remove(0)` sem perceber o deslocamento de `ArrayList`; para 52 cartas não é grave, mas a escolha deve ser consciente.
- Devolver a lista para Blackjack remover diretamente.
- Clonar uma Carta na compra e deixar a original no baralho.
- Retornar null e espalhar verificações tardias.

Pronto quando

- ☐ Comprar remove exatamente uma referência.
- ☐ A lista interna continua inacessível para mutação externa.
- ☐ O comportamento de vazio é explícito e testado.
- ☐ Embaralhar pertence ao Baralho.
- ☐ Os testes detectariam um baralho com 52 ases iguais.

Explique em voz alta

- Por que embaralhar não faz sentido sem escolher uma instância de Baralho?
- Qual objeto conhece as cartas restantes?
- Como provar que nenhuma carta foi duplicada?
- O que deveria acontecer na compra 53?

MODELO VISUAL

A transferência de uma carta



comprar() remove e devolve; adicionar() conserva a referência

A carta não deve continuar simultaneamente nas duas coleções.

Figura 3. Uma referência sai do baralho e passa a ser conservada pela mão.

A colaboração recomendada é uma sequência de mensagens: Blackjack pede uma carta ao Baralho; Baralho remove e devolve; Blackjack entrega à Mão; Mão conserva. A mão não conhece o baralho e o baralho não conhece participantes.

Invariante de conservação

CONTA DAS CARTAS

Durante uma rodada sem descarte, a soma das cartas restantes no baralho e das cartas nas mãos deve permanecer igual a 52. Depois da distribuição inicial para um jogador e uma banca: $48 + 2 + 2 = 52$.

Experimento de referência

1. Guarde a carta retornada por comprar em uma variável local.
2. Adicione exatamente essa referência a uma Mão.
3. Verifique que o tamanho do baralho diminuiu e o da mão aumentou.
4. Se existir uma política de igualdade, diferencie `assertSame` de `assertEquals`.
5. Explique por que `addAll` ou `add` copiam referências, não clonam objetos.

CAPÍTULO F4

F4

Transformar Blackjack em coordenador

O construtor deve criar a partida ou apenas receber e validar seus participantes?

OBJETIVO DA FASE

Eliminar inicializações redundantes e fazer Blackjack coordenar objetos por comportamentos públicos.

O que o código atual revela

- Blackjack recebe Baralho, Jogador e Dealer, mas cria versões temporárias antes do corpo do construtor.
- O campo nome permanece null e duplica uma informação que pertence ao Jogador.
- Os campos têm visibilidade de pacote e não são finais.
- O método embaralhar está vazio, estático e na classe errada.
- App cria a variável jogo, mas não chama nenhuma operação.

Conceitos que você está praticando

- Injeção de dependência manual.
- Construtor e invariantes.
- Orquestração versus posse de dados.
- Baixo acoplamento por mensagens.
- Efeitos explícitos.

PARE E PENSE

O construtor deve criar a partida ou apenas receber e validar seus participantes?

Decisões antes do código

1. Escolha definitivamente: Blackjack recebe suas dependências ou as cria; a recomendação é recebê-las.
2. Defina quais dependências jamais podem ser null.
3. Decida qual método inicia uma rodada.
4. Defina se a distribuição inicial alterna jogador e banca.
5. Planeje como testar com uma ordem de cartas previsível.

Missão prática

- ☐ Remover o campo nome.
- ☐ Remover inicializadores que constroem Baralho, Jogador e Dealer.
- ☐ Tornar as três dependências privadas e finais.
- ☐ Validar dependências no construtor.
- ☐ Criar uma operação explícita de distribuição inicial.
- ☐ Pedir cartas por meio de Baralho.comprar().
- ☐ Entregar cartas por meio do comportamento de Mao.
- ☐ Remover o método embaralhar de Blackjack.

Contratos sugeridos

Operação	Responsabilidade	Não deve fazer
Blackjack(...)	Conservar dependências obrigatórias válidas.	Criar substitutos temporários ou começar a rodada silenciosamente.
distribuirCartasIniciais()	Transferir duas cartas para cada participante.	Manipular as listas internas diretamente.
iniciarRodada()	Coordenar preparação segundo a sequência definida.	Ler Scanner ou imprimir interface dentro das entidades.

Testes-alvo

Cenário	Evidência observável
Construção	Não cria um segundo baralho nem participantes secretos.
Dependência null	Construção falha perto da causa.
Distribuição inicial	Jogador e Dealer recebem duas cartas cada.
Conservação	Baralho termina com 48 cartas.
Ordem	Com baralho preparado, a alternância é previsível.
Identidade	As cartas entregues são as mesmas retiradas.

Armadilhas previsíveis

- Fazer o construtor executar toda a partida.
- Criar novos participantes dentro de distribuir e ignorar os campos.
- Pedir a lista interna do Baralho para removê-la externamente.
- Misturar `Scanner` e `System.out` com regras de domínio.
- Testar distribuição com embaralhamento real e aceitar testes intermitentes.

Pronto quando

- ☐ Construir Blackjack não aloca outro baralho.
- ☐ As dependências recebidas permanecem as utilizadas.
- ☐ A distribuição inicial é explícita e determinística em teste.
- ☐ Depois da distribuição, $48 + 2 + 2 = 52$.
- ☐ Blackjack coordena; Baralho e Mao protegem seus próprios estados.

Explique em voz alta

- Por que receber dependências facilita testes?
- Qual é a diferença entre coordenar uma transferência e manipular duas listas?
- Por que nome pertence ao Jogador?
- Que efeitos devem acontecer apenas quando iniciarRodada for chamado?

CAPÍTULO F5

F5

Calcular pontuação no lugar correto

Por que uma Carta isolada não consegue decidir se o ás vale 1 ou 11?

OBJETIVO DA FASE

Fazer Mao interpretar o conjunto de cartas sem alterar a identidade de nenhuma Carta.

O que o código atual revela

- Carta conserva identidade e naipe, mas ainda não oferece valor-base tipado.
- Mao ainda não calcula total, estouro ou blackjack natural.
- O ás depende das outras cartas; sua interpretação é contextual.
- Armazenar um campo total duplicaria informação derivável da coleção.

Conceitos que você está praticando

- Valor derivado.
- Responsabilidade baseada em contexto.
- Algoritmo para múltiplos ases.
- Mão macia.
- Blackjack natural versus total 21.

PARE E PENSE

Por que uma Carta isolada não consegue decidir se o ás vale 1 ou 11?

Decisões antes do código

1. Decida se ValorCarta informa o valor-base do ás como 11.
2. Defina a regra: somar ases como 11 e ajustar enquanto necessário.
3. Defina a política de blackjack natural.
4. Decida se o total será sempre recalculado ou terá cache cuidadosamente invalidado; recomendo recalculá-lo agora.
5. Defina nomes claros para estourou, temVinteEUm, temBlackjackNatural e ehMacia.

Missão prática

- ☐ Somar cartas numéricas normalmente.
- ☐ Somar J, Q e K como 10.
- ☐ Contar inicialmente cada ás como 11.
- ☐ Enquanto o total exceder 21 e houver ás flexível, subtrair 10.
- ☐ Criar consultas derivadas sem campos booleanos duplicados.
- ☐ Não alterar Carta durante o cálculo.
- ☐ Cobrir vários ases em testes separados.

Contratos sugeridos

Operação	Responsabilidade	Não deve fazer
Mao.calcularPontuacao()	Derivar o melhor total possível das cartas atuais.	Alterar o rank ou guardar total desatualizado.
Mao.estourou()	Responder se o total é maior que 21.	Manter um campo independente do cálculo.
Mao.temBlackjackNatural()	Verificar total 21 com exatamente duas cartas.	Chamar todo total 21 de blackjack.
Mao.ehMacia()	Indicar se algum ás ainda conta como 11.	Decidir a política da banca.

Testes-alvo

Cenário	Evidência observável
10 + Q	Total 20.
A + 9	Total 20 e mão macia.
A + K	Total 21 e blackjack natural.
A + A	Total 12.
A + A + 9	Total 21.
A + 6 + 10	Total 17 após ajustar o ás.
A + A + A + 8	Total 21.
7 + 7 + 7	Total 21, mas não blackjack natural.
K + Q + 2	Total 22 e estouro.

Armadilhas previsíveis

- Transformar o campo do ás de 11 para 1 durante o cálculo.
- Ajustar apenas um ás e falhar com três ases.
- Guardar o total e esquecer de atualizá-lo ao receber carta.
- Colocar a lógica completa em Carta.
- Tratar 21 com três cartas como blackjack natural.

Pronto quando

- ☐ Todos os cenários de ás passam.
- ☐ O cálculo não modifica as cartas.
- ☐ O total sempre reflete a coleção atual.
- ☐ Blackjack natural exige exatamente duas cartas.
- ☐ Mão macia é distinguível para a política da banca.

Explique em voz alta

- Qual objeto vê todas as cartas necessárias para ajustar o ás?
- Por que pontuação é valor derivado?
- Em A + A + 9, quantos ases ainda podem valer 11?
- Por que 7 + 7 + 7 não é blackjack natural?

LABORATÓRIO DE REGRAS

Teste manual de múltiplos ases

Mão	Total	Macia?	Raciocínio
A + 6	17	Sim	Um ás permanece 11.
A + 9	20	Sim	Um ás permanece 11.
A + K	21	Sim	Natural com duas cartas.
A + A	12	Sim	11 + 1.
A + A + 9	21	Sim	11 + 1 + 9.
A + 6 + 10	17	Não	O ás precisa cair de 11 para 1.
A + A + 10 + K	22	Não	Mesmo com ambos valendo 1, estoura.
A + A + A + 8	21	Sim	11 + 1 + 1 + 8.

Ficha de previsão

- ☐ Antes de executar, escrevi o total esperado.
- ☐ contei quantos ases foram inicialmente somados como 11.
- ☐ Ajustei um ás por vez somente enquanto o total excedia 21.
- ☐ Verifiquei separadamente total, estouro, mão macia e blackjack natural.
- ☐ Se o teste falhou, descrevi qual regra estava ausente.

INVARIANTE ÚTIL

A pontuação escolhida deve ser a maior soma válida que não ultrapassa 21; se todas ultrapassarem, deve ser a menor soma possível após ajustar todos os ases necessários.

FASE F6

F6

Modelar turnos e resultado da rodada

Que ações são válidas em cada estado, e qual regra decide cada transição?

OBJETIVO DA FASE

Modelar uma rodada explícita: estados coerentes, precedência de naturais, resultado separado do motivo e nenhuma ação depois do fim.

O que o projeto atual revela

- Não existe turno humano, turno da banca nem cálculo de resultado.
- O objeto Blackjack é construído, mas não recebe uma operação de início.
- Soft 17, naturais, carta oculta e nova rodada ainda não estão registrados no README ou nos testes do projeto.
- App não deve decidir vencedor nem recalculer pontuação.

Modelo escolhido

Estado	Significado
CRIADA	Dependências válidas; nenhuma carta distribuída.
CARTAS_DISTRIBUIDAS	Duas cartas por participante; naturais serão avaliados.
TURNO_DO_JOGADOR	Pedir ou parar; 21 e estouro encerram decisões.
TURNO_DA_BANCA	Carta oculta revelada; compras seguem S17.
FINALIZADA	Resultado e motivo imutáveis; compras rejeitadas.

Conceitos praticados

- Máquina de estados, comando versus consulta e pré-condições.
- Enum de estado; resultado e motivo como valores distintos.
- Política da banca separada da interface e efeitos explícitos.

PARE E PENSE

Se o jogador chegou a 21, qual evento avança o estado sem pedir outra entrada? Se ambos nasceram com natural, por que nenhum turno deve começar?

Decisões e contratos dos turnos

Decisões antes do código

1. Use exatamente os cinco estados definidos na página anterior.
2. Avalie ambos os naturais em `CARTAS_DISTRIBUIDAS`, antes do turno humano.
3. Ao pedir: transfira uma carta; 21 avança para a banca e estouro finaliza.
4. Ao parar: revele a carta da banca e avance para `TURNO_DA_BANCA`.
5. A banca para em todo 17 ou mais, inclusive soft 17.
6. Nova rodada não reabre `FINALIZADA`; App cria novos objetos de rodada.

Missões práticas

- ☐ Criar enums `EstadoRodada`, `Resultado` e `MotivoResultado`.
- ☐ Implementar `iniciar`, `pedirCartaJogador` e `pararJogador`.
- ☐ Revelar a carta somente pela visão de apresentação.
- ☐ Executar a banca automaticamente e determinar o desfecho sem imprimir.
- ☐ Rejeitar comandos incompatíveis com o estado atual.

Contratos sugeridos

Operação	Pré-condição	Pós-condição
<code>iniciarRodada</code>	<code>CRIADA</code>	Distribui; avalia naturais; avança ou finaliza.
<code>pedirCartaJogador</code>	<code>TURNO_DO_JOGADOR</code>	Transfere uma; 21 avança; estouro finaliza.
<code>pararJogador</code>	<code>TURNO_DO_JOGADOR</code>	Avança para <code>TURNO_DA_BANCA</code> .
<code>executarTurnoDealer</code>	<code>TURNO_DA_BANCA</code>	Revela e compra até a política mandar parar.
<code>determinarResultado</code>	Rodada resolvida	Produz <code>Resultado</code> e <code>Motivo</code> , sem imprimir.
<code>getVisaoDealer</code>	Qualquer estado válido	Oculto ou revela sem expor lista mutável.

REGRA DE API

App traduz entrada em comandos e apresenta consultas. Nenhum setter permite alterar estado, resultado ou cartas por fora.

Testes-alvo de turnos e resultado

Cenário	Evidência observável
Ambos naturais	EMPATE / AMBOS_NATURAIS; nenhum turno inicia.
Só jogador natural	VITORIA_JOGADOR / BLACKJACK_NATURAL.
Só banca natural	VITORIA_BANCA / BLACKJACK_NATURAL.
Jogador pede	Baralho -1 e Mao +1 quando o estado permite.
Jogador chega a 21	Turno humano termina sem nova pergunta.
Jogador estoura	Finaliza; banca não compra cartas extras.
Jogador para	Avança para TURNO_DA_BANCA e revela a carta.
Banca em 16	Compra.
Banca em hard 17	Para.
Banca em soft 17	Para pela regra S17.
Banca estoura	Jogador vence por ESTOURO.
Totais iguais	EMPATE / TOTAIS_IGUAIS.
Natural vs 21 comum	Natural recebe precedência.
FINALIZADA	Pedir, parar ou executar banca é rejeitado.

Armadilhas previsíveis

- Misturar Resultado com Motivo ou representar ambos por texto livre.
- Executar a banca após estouro do jogador.
- Oferecer pedir quando o total já é 21.
- Remover a carta oculta da Mao em vez de ocultar apenas sua exibição.
- Reutilizar estado FINALIZADA para nova rodada sem reset completo.

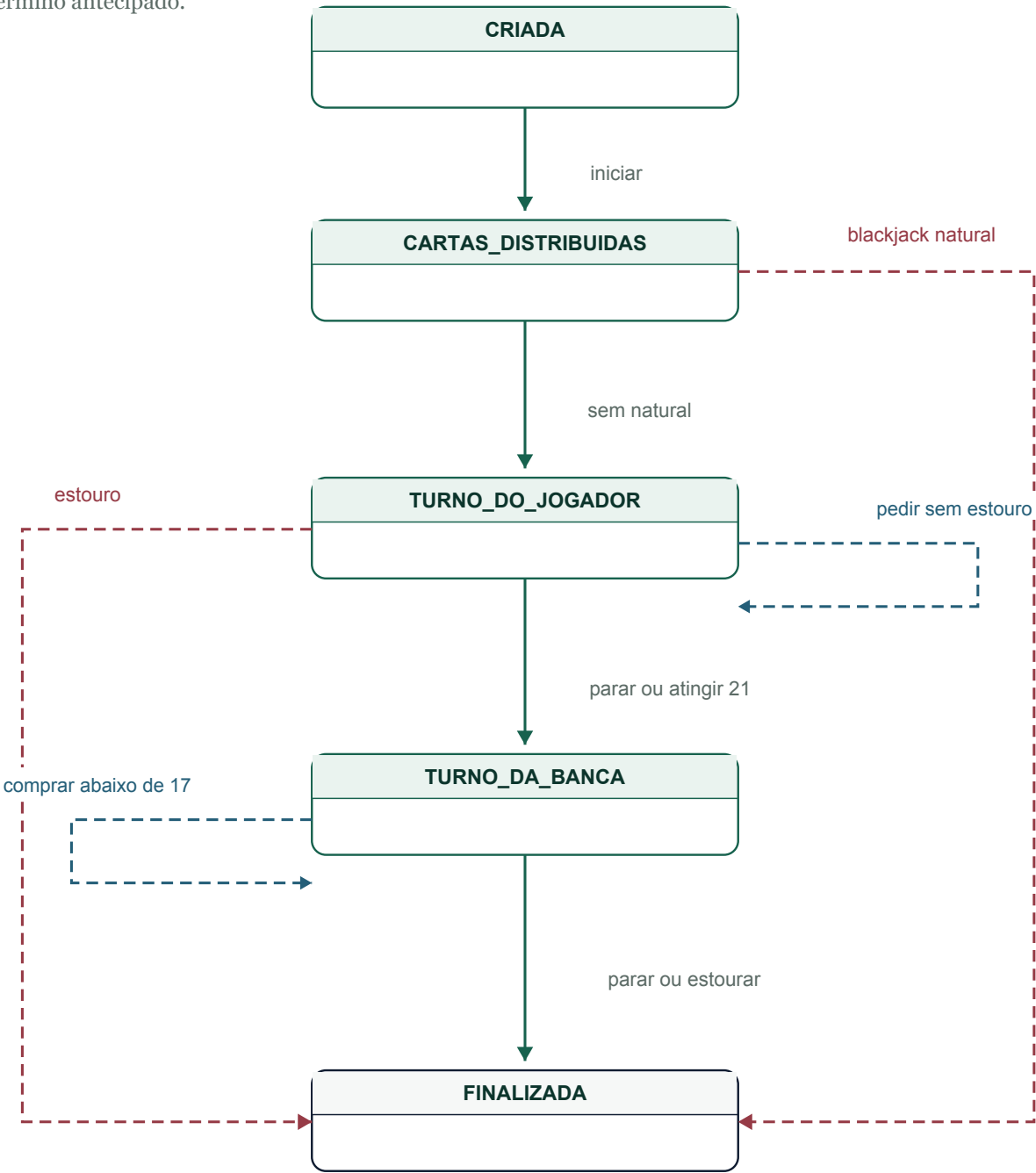
PRONTO QUANDO

Cada transição possui teste; naturais precedem turnos; S17 está explícita; resultado e motivo são consultáveis; nenhum comando altera uma rodada finalizada.

MODELO VISUAL

Máquina de estados mínima

Um único estado FINALIZADA recebe todos os términos. Linha sólida indica o fluxo normal; tracejada indica término antecipado.



REGRA DE CONSISTÊNCIA

Use exatamente os mesmos nomes de estado no código, nos testes, no UML e nas tabelas. Não crie duas caixas FINALIZADA para resultados diferentes; o resultado é dado do estado final.

CAPÍTULO F7

F7

Conectar o domínio à interface

Como App pode conduzir a rodada sem receber listas internas nem recalculas regras?

OBJETIVO DA FASE

Fazer a interface cuidar apenas de entrada, saída e tradução das escolhas do usuário para mensagens ao jogo.

O que o código atual revela

- O menu não mostra o, embora o laço encerre com o.
- Opções 2, 3 e 4 aparecem, mas são ignoradas.
- `next()` lê apenas um token; nomes com espaços deixam conteúdo que quebra a próxima leitura numérica.
- Entrada não numérica provoca exceção.
- O objeto jogo é criado e imediatamente abandonado sem receber uma operação.
- O nome `welcomeBlackjack` está grafado incorretamente e mistura idiomas.

Conceitos que você está praticando

- Separação entre interface e domínio.
- Parsing e validação de entrada.
- Loop de interação.
- switch sem fall-through.
- Testabilidade sem Scanner.

PARE E PENSE

Como App pode conduzir a rodada sem receber listas internas nem recalculas regras?

Decisões antes do código

1. Decida se o menu mostrará apenas Blackjack e sair durante o MVP.
2. Escolha uma estratégia: ler linhas completas e converter a opção.
3. Defina mensagens para opção inválida e jogo ainda não implementado.
4. Defina quais dados de estado Blackjack expõe para apresentação.
5. Planeje o ciclo: iniciar, mostrar mãos, pedir/parar, resultado, repetir/sair.

Missão prática

- ☐ Mostrar explicitamente 0) Sair.
- ☐ Tratar texto não numérico sem encerrar o programa.
- ☐ Aceitar nome completo e rejeitar nome em branco.
- ☐ Adicionar comportamento para opções indisponíveis ou removê-las do menu.
- ☐ Chamar a operação que inicia a rodada.
- ☐ Traduzir pedir/parar em mensagens ao objeto Blackjack.
- ☐ Exibir resultado retornado pelo domínio.
- ☐ Manter cálculo de pontos e vencedor fora de App.

Contratos sugeridos

Operação	Responsabilidade	Não deve fazer
lerOpcao()	Converter uma linha válida em opção.	Executar regras de Blackjack.
jogarBlackjack(nome)	Montar dependências e conduzir a interface.	Calcular pontuação diretamente.
mostrarEstado(jogo)	Apresentar dados públicos do domínio.	Receber a lista mutável de cartas.
main	Controlar o ciclo da aplicação.	Virar um objeto global compartilhado por static.

Testes-alvo

Cenário	Evidência observável
Entrada o	Programa encerra de forma clara.
Entrada não numérica	Mensagem útil e nova tentativa.
Nome com espaços	Nome completo é conservado.
Opção indisponível	Usuário recebe informação, sem silêncio.
Rodada completa	É possível pedir/parar e ver resultado.
Nova rodada	Estado anterior não vaza para a próxima.

Armadilhas previsíveis

- Misturar `nextInt()`, `next()` e `nextLine()` sem entender o buffer.
- Duplicar o cálculo de pontos em App para imprimir.
- Deixar opções inexistentes no menu sem resposta.
- Fechar `System.in` dentro de um método auxiliar.
- Usar variáveis `static` para compartilhar jogo entre métodos.

Pronto quando

- ☐ A opção de saída está visível e encerra o programa corretamente.
- ☐ Entrada inválida não encerra o processo.
- ☐ Nome completo é aceito conforme a decisão.
- ☐ App chama comportamentos; não manipula coleções.
- ☐ Uma rodada manual termina e pode ser repetida.

Explique em voz alta

- Que parte do programa conhece Scanner?
- Que parte sabe quem venceu?
- Como evitar que um nome com espaço quebre a próxima opção?
- Quais informações a interface precisa consultar sem quebrar encapsulamento?

CAPÍTULO F8

F8

Usar testes, UML e README como especificação

Se a implementação mudar amanhã, quais artefatos revelarão imediatamente uma regressão ou divergência?

OBJETIVO DA FASE

Transformar verificação e documentação em trilhas permanentes, não em tarefas deixadas para o final.

O que o código atual revela

- Os três testes atuais cobrem quantidade inicial, conservação/normalização da Carta e naipe nulo.
- Todos os testes ainda estão em AppTest.
- O PUML omite delimitadores e descreve métodos removidos ou em classes erradas.
- README documenta comandos Maven, mas não o domínio ou o estado de implementação.

Conceitos que você está praticando

- Arrange, Act, Assert.
- Teste de comportamento público.
- Determinismo e aleatoriedade controlada.
- UML descritivo versus modelo-alvo.
- Documentação de decisões.

PARE E PENSE

Se a implementação mudar amanhã, quais artefatos revelarão imediatamente uma regressão ou divergência?

Decisões antes do código

1. Defina uma classe de teste por unidade principal.
2. Escreva cenários com nome que descreve causa e resultado.
3. Não teste métodos privados diretamente; prove seus efeitos pela API.
4. Mantenha dois diagramas enquanto necessário: estado atual e alvo.
5. Atualize README ao decidir uma regra, não meses depois.

Missão prática

- ☐ Criar CartaTest, BaralhoTest, MaoTest, JogadorTest e BlackjackTest conforme surgirem comportamentos.
- ☐ Escrever teste antes ou junto da menor implementação.
- ☐ Usar Dado/Quando/Então para cenários de domínio.
- ☐ Corrigir @startuml e @enduml.
- ☐ Remover do PUMML métodos que não existem.
- ☐ Adicionar Naipes e o futuro ValorCarta.
- ☐ Documentar escopo e soft 17 no README.
- ☐ Executar `mvn clean test` em cada marco.

Contratos sugeridos

Operação	Responsabilidade	Não deve fazer
Teste unitário	Provar uma regra por API pública.	Depender de ordem aleatória não controlada.
blackjack.puml	Representar fatos atuais ou alvo claramente rotulado.	Misturar os dois sem aviso.
README	Ensinar a executar e registrar decisões.	Virar cópia extensa da implementação.

Especificação viva: testes, PUMML e README

Cada artefato responde uma pergunta diferente e deve ser atualizado no mesmo marco da mudança.

Estado atual comprovado

- Três testes passam: tamanho inicial, normalização/conservação de Carta e naipe nulo.
- Os testes ainda estão concentrados em AppTest.
- O PUMML omite delimitadores, Naipe e membros atuais; também descreve métodos removidos.
- O README ensina Maven, mas não registra regras, escopo ou estado de implementação.

Contrato de cada artefato

Artefato	Deve provar	Não deve
Teste	Provar comportamento público, limites e erros de uma regra.	Não depender de aleatoriedade real.
PUMML atual	Mostrar membros e referências que existem hoje.	Não incluir futuro sem rótulo.
PUMML alvo	Registrar a arquitetura pretendida e suas multiplicidades.	Não se passar por snapshot atual.
README	Ensinar a executar e registrar S17, naturais, carta oculta e nova rodada.	Não copiar corpos de métodos.

Missões da fase

- ☐ Separar CartaTest, BaralhoTest, MaoTest, JogadorTest, DealerTest e BlackjackTest conforme surgirem comportamentos.
- ☐ Usar Dado/Quando/Então ou Arrange/Act/Assert e nomes que revelem causa e efeito.
- ☐ Testar naturais, soft/hard 17, carta oculta, estado final e nova rodada.
- ☐ Corrigir @startuml/@enduml; adicionar Naipe, ValorCarta e multiplicidades.
- ☐ Registrar no README toda regra congelada antes de implementar o fluxo.

EVIDÊNCIA MÍNIMA

Uma fase termina apenas com comportamento observável, teste que falha pelo motivo certo, diagrama coerente e decisão de regra documentada.

CAPÍTULO 04

04

Matriz completa de testes

Implemente por fase. A matriz inclui regras felizes, limites, erros e transições do MVP fechado.

Unidade	Cenário	Prova
CartaTest	13 ranks e 4 naipes	Dados conservados; texto legível.
CartaTest	null, branco e rank impossível	Objeto inválido não nasce.
CartaTest	igualdade e hash	Política rank+naipe coerente.
BaralhoTest	52 combinações	13/naipe, 4/rank, nenhuma null.
BaralhoTest	comprar e esgotar	Remove a mesma referência; vazio segue contrato.
BaralhoTest	embaralhar	Preserva tamanho e multiconjunto.
MaoTest	vazia, adicionar, null	Estado persiste e contrato é protegido.
MaoTest	independência	Duas mãos não compartilham lista.
MaoTest	figuras e ases	Melhor total; A+A, A+A+9, A+A+A+8.
MaoTest	macia, natural, estouro	Predicados derivados corretos.
JogadorTest	nome e Mao	Nome inválido rejeitado; Mao própria.
DealerTest	Mao própria	Não compartilha Mao com jogador.
BlackjackTest	dependências null	Construção falha perto da causa.
BlackjackTest	distribuição	2 + 2 e 48 restantes; identidade preservada.
BlackjackTest	baralho insuficiente	Falha sem distribuição parcial.
BlackjackTest	naturais	Ambos, só jogador, só banca; precedência.
BlackjackTest	pedir, 21 e estouro	Transições e compras corretas.
BlackjackTest	parar e banca	Hard 16 compra; hard/soft 17 param.
BlackjackTest	resultado e motivo	Estouro do jogador/banca, natural, maior total e empate.
BlackjackTest	visibilidade	Carta existe na Mao e só a exibição oculta.
BlackjackTest	FINALIZADA	Comandos posteriores são rejeitados.
Integração	nova rodada	Novos objetos; nome preservado; sem estado antigo.
Aceitação	terminal	Entrada inválida, nome completo, rodada e saída.

UNICIDADE LÓGICA

HashSet<Carta> só prova 52 combinações depois de equals/hashCode por rank+naipe. Antes disso, compare chaves compostas rank+naipe.

Modelo Dado / Quando / Então

FICHA DE TESTE

Dado que:
[estado inicial relevante]

Quando:
[uma única ação pública]

Então:
[resultado observável]

Qual erro este teste impediria?
[uma frase concreta]

Arrange, Act, Assert

Parte	Pergunta	Sinal de problema
Arrange	Que objetos e estado inicial o cenário exige?	Preparação maior que o comportamento testado.
Act	Qual única mensagem dispara o comportamento?	Várias ações importantes no mesmo teste.
Assert	Qual efeito público prova a regra?	Inspeção de campo privado ou muitas causas de falha.

Aleatoriedade sem testes frágeis

- Não afirme que embaralhar necessariamente muda a ordem.
- Verifique que o tamanho e o conjunto de cartas foram preservados.
- Para uma distribuição previsível, use um baralho preparado ou uma fonte de Random controlada.
- Evite repetir um teste aleatório até passar; isso esconde defeitos.

TESTE DE QUALIDADE

Um teste bom falha por uma regra compreensível. Se ele pode falhar apenas porque o sorteio foi raro, ele não é uma especificação confiável.

Testes que parecem bons, mas não bastam

Teste superficial	O que deixa escapar	Como fortalecer
Baralho tem 52	Não detecta 52 cartas iguais.	Verificar combinações e distribuição.
Pontuação A + K	Não detecta erro com múltiplos ases.	Cobrir A + A e A + A + 9.
Jogador tem Mao	Pode ser a mesma instância do Dealer.	Usar assertNotSame e alterar uma.
Shuffle mudou ordem	Pode falhar legitimamente.	Testar preservação ou injetar aleatoriedade.
App imprime resultado	Pode ocultar regra duplicada na interface.	Testar resultado no domínio.

Ordem de execução recomendada

1. Rode somente a classe de teste da fase durante o ciclo curto.
2. Rode toda a suíte antes de considerar a fase pronta.
3. Rode `mvn clean test` no marco para eliminar resíduos de compilação.
4. Se um teste antigo falhar, descubra se a regra mudou ou se houve regressão.
5. Nunca altere a expectativa apenas para fazer o teste passar sem justificar a nova regra.

CAPÍTULO 05

05

Laboratório de UML

O diagrama deve mostrar quem conhece quem, quais referências são conservadas e onde vive cada responsabilidade.

Problemas atuais de blackjack.puml

- Faltam `@startuml` e `@enduml`.
- Carta ainda mostra `criarNaipe`, método já removido.
- Baralho mostra chamadas como membros e um método `gerarBaralho` inexistente.
- Blackjack mostra `playBlackjack`, que pertence a App.
- Naipe não aparece.
- Jogador aparece possuindo `Mao`, mas o código atual ainda perde a referência local.
- `Mao` aparece contendo cartas, mas ainda não possui campo.
- As visibilidades não refletem os campos `package-private` atuais.

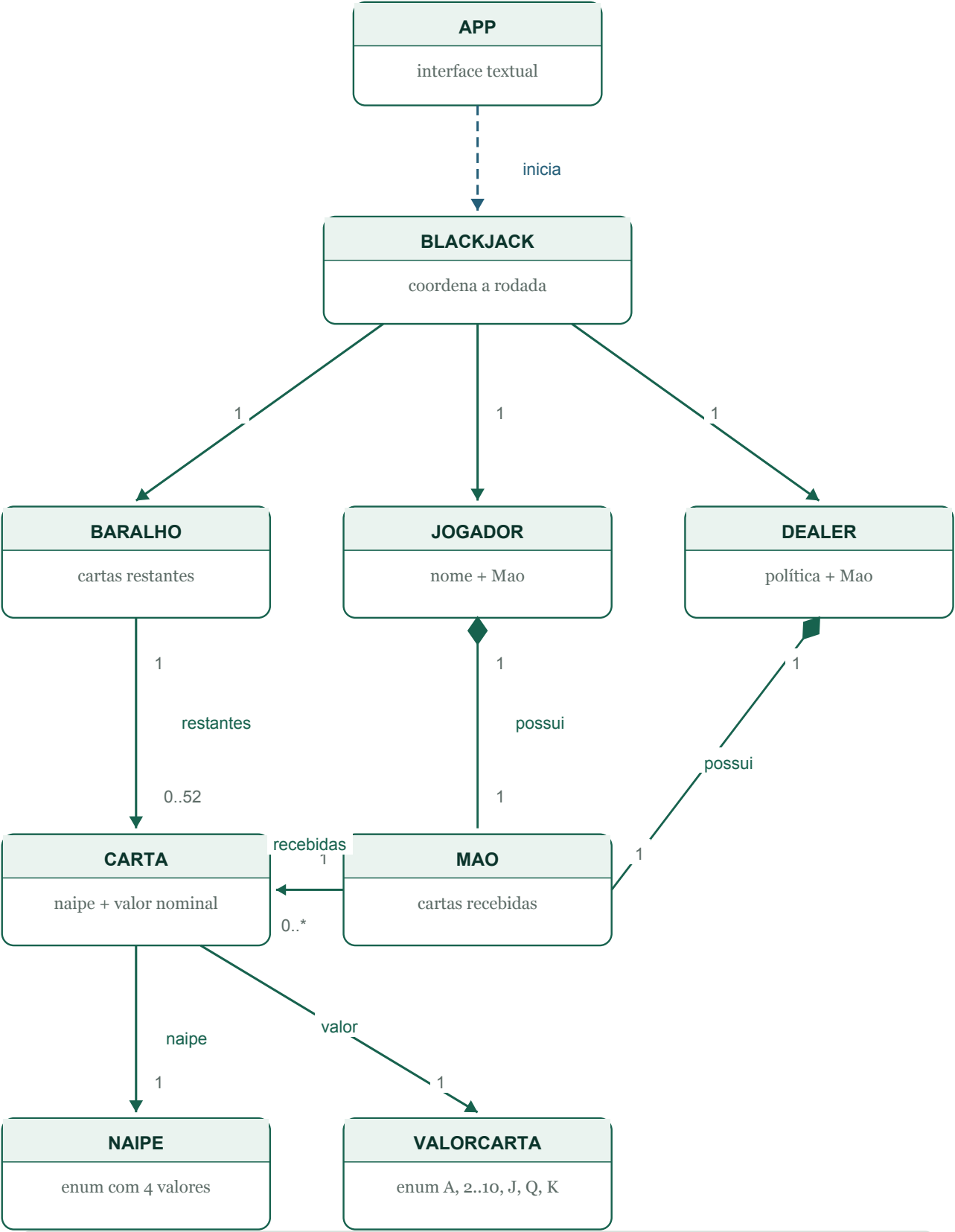
DUAS FOTOGRAFIAS

Enquanto o projeto muda, mantenha um diagrama 'como está' e outro 'modelo-alvo'. Misturá-los em uma única figura transforma intenções futuras em afirmações falsas sobre o código atual.

MODELO-ALVO

UML conceitual do MVP

O diagrama distingue dependência, associação navegável e composição sem depender apenas de cor.



LEGENDA UML

Tracejada: dependência | seta: associação navegável
Losango cheio: composição | rótulos 1, 0..*: multiplicidades

Relações e multiplicidades do modelo-alvo

Relação	Multiplicidade	Leitura
App ..> Blackjack	uso temporário	App monta e conduz uma rodada.
Blackjack -> Baralho	1 para 1	Conserva o baralho da rodada.
Blackjack -> Jogador	1 para 1	Conserva o participante humano.
Blackjack -> Dealer	1 para 1	Conserva a banca.
Jogador *-- Mao	1 para 1	Composição escolhida no MVP sem split.
Dealer *-- Mao	1 para 1	Mao exclusiva da banca.
Baralho --> Carta	1 para 0..52	Somente cartas ainda disponíveis.
Mao --> Carta	1 para 0..*	Somente cartas recebidas.
Carta --> Naipe	cada Carta tem 1	Tipo fechado com quatro valores.
Carta --> ValorCarta	cada Carta tem 1	Tipo fechado com treze ranks.

RESTRIÇÃO DE CONSERVAÇÃO

A mesma instância de Carta não pertence simultaneamente ao Baralho e a uma Mao. Comprar remove da origem antes de adicionar ao destino.

PLANTUML - ESQUELETO ALVO

```
@startuml
title Blackjack - MODELO-ALVO
enum Naipe
enum ValorCarta
class Carta
class Baralho
class Mao
class Jogador
class Dealer
class Blackjack
class App

App ..> Blackjack
Blackjack "1" --> "1" Baralho
Blackjack "1" --> "1" Jogador
Blackjack "1" --> "1" Dealer
Jogador "1" *-- "1" Mao
Dealer "1" *-- "1" Mao
Baralho "1" --> "0..52" Carta : restantes
Mao "1" --> "0..*" Carta : recebidas
Carta --> "1" Naipe
Carta --> "1" ValorCarta
@enduml
```

RÓTULO OBRIGATÓRIO

Mantenha outro arquivo ou outra figura para o estado ATUAL. Nunca apresente relações futuras como se já existissem no código.

Checklist de sincronização do diagrama

- ☐ O arquivo possui delimitadores PlantUML.
- ☐ Todas as classes e enums atuais aparecem.
- ☐ Atributos têm nome, tipo e visibilidade.
- ☐ Métodos têm nome, parâmetros e retorno.
- ☐ Métodos removidos desapareceram do diagrama.
- ☐ Métodos não foram colocados na classe errada.
- ☐ Relações correspondem a referências reais.
- ☐ Multiplicidades refletem o escopo atual do MVP.
- ☐ Composição é usada somente quando a afirmação de posse é intencional.
- ☐ O título declara se o diagrama é atual ou alvo.
- ☐ O arquivo renderiza sem erro.

Perguntas de revisão

- Qual lado do relacionamento conserva a referência?
- A seta descreve uma dependência temporária ou uma associação persistente?
- O diamante está no lado do todo?
- Split mudaria a multiplicidade entre Jogador e Mao?
- Há duplicação suficiente para justificar uma classe Participante agora?

CAPÍTULO 06

06

Contratos do modelo-alvo

Cada assinatura promete uma pré-condição observável e uma pós-condição coerente com o MVP.

Operação	Pré-condição	Pós-condição
ValorCarta.getValorBase()	Valor válido	Base numérica; ás segue contextual.
Mao.adicionar(Carta)	Carta não null	Quantidade +1; mesma referência.
Mao.calcularPontuacao()	Coleção válida	Melhor total com todos os ases.
Mao.estourou()	Mao válida	true somente quando total > 21.
Mao.temBlackjackNatural()	Mao válida	21 com exatamente duas cartas.
Mao.ehMacia()	Mao válida	Indica ás ainda contado como 11.
Baralho.embaralhar()	Baralho válido	Mesmo multiconjunto; ordem possível.
Baralho.comprar()	Baralho não vazio	Remove e devolve uma Carta.
Dealer.deveComprar()	Mao pontuável	true abaixo de 17; false em todo 17+.
Blackjack.distribuirCartasIniciais()	Estado CRIADA; 4 cartas	2+2; 48 restantes; avalia naturais.
Blackjack.pedirCartaJogador()	Turno humano	Transfere; 21 avança; estouro finaliza.
Blackjack.pararJogador()	Turno humano	Avança para TURNO_DA_BANCA.
Blackjack.executarTurnoDealer()	Turno da banca	Revela, aplica S17 e finaliza.
Blackjack.determinarResultado()	Rodada resolvida	Resultado e MotivoResultado separados.
App.lerOpcao()	Linha disponível	Opção válida ou nova tentativa.

DOIS VALORES, DUAS PERGUNTAS

Resultado responde quem venceu: VITORIA_JOGADOR, VITORIA_BANCA ou EMPATE. MotivoResultado responde por quê: natural, estouro, maior total ou igualdade.

Invariantes por classe

Classe	Invariante
Carta	Rank e naipe sempre válidos; identidade não muda.
Mao	Coleção existe, não contém null e representa todas as cartas recebidas.
Baralho	Coleção representa apenas cartas restantes; compra não duplica.
Jogador	Nome válido e uma mão própria no MVP.
Dealer	Uma mão própria e política de compra conhecida.
Blackjack	Dependências válidas; estado permite apenas operações coerentes.
App	Entrada inválida é tratada; regras não são duplicadas na interface.

Sinais de encapsulamento saudável

- Campos são privados por padrão.
- Operações expressam verbos do domínio: comprar, adicionar, calcular, parar.
- Coleções internas não são devolvidas para mutação externa.
- Consultas derivam estado em vez de duplicá-lo em booleans.
- Construtores impedem objetos parcialmente válidos.
- A interface não sabe como a pontuação é calculada.

ENCAPSULAMENTO NÃO É

Colocar private em tudo e gerar getters/setters automaticamente. Encapsular é proteger invariantes por meio de uma API que só permite ações válidas.

CAPÍTULO 07

07

Definição de pronto do MVP

Marque somente quando houver teste, execução reproduzível, código revisado ou diagrama renderizado.

Domínio e estado

- ☐ Carta aceita apenas 13 ranks e 4 naipes.
- ☐ Carta é imutável, legível e tem igualdade coerente.
- ☐ Baralho prova 52 combinações únicas.
- ☐ Comprar remove a mesma referência.
- ☐ Cada participante conserva Mao própria.
- ☐ Coleções internas não escapam mutáveis.
- ☐ Pontuação cobre figuras e múltiplos ases.
- ☐ Mão macia, natural e estouro são derivados.
- ☐ Dependências obrigatórias são privadas e finais.
- ☐ Não há objetos criados apenas para substituição.

Fluxo e regras

- ☐ Distribuição inicial produz 2 + 2 e 48 restantes.
- ☐ Carta da banca é oculta só na apresentação.
- ☐ Ambos os naturais são avaliados antes dos turnos.
- ☐ Natural vence 21 comum; ambos naturais empatam.
- ☐ Jogador pode pedir/parar; 21 encerra seu turno.
- ☐ Estouro humano finaliza sem turno extra da banca.
- ☐ Banca compra abaixo de 17 e para em soft 17.
- ☐ Resultado e Motivo permanecem separados.
- ☐ FINALIZADA rejeita novos comandos.
- ☐ Nova rodada usa novos objetos e preserva só o nome.

Qualidade, documentação e evidência

- ☐ mvn clean test passa; testes cobrem sucesso, limite, erro e transição.
- ☐ README registra S17, precedência, carta oculta, nova rodada, escopo e comandos Maven.
- ☐ PUML atual e alvo estão rotulados; enums, membros, multiplicidades e setas correspondem ao modelo.
- ☐ Entrada inválida não encerra o programa; nome completo é aceito; uma rodada manual termina sem exceção.

CRITÉRIO OBJETIVO

'Parece funcionar' não marca item. Registre o teste, o comando executado ou a observação manual que prova cada afirmação.

Qualidade técnica

- ☐ mvn clean test termina com BUILD SUCCESS.
- ☐ Os testes estão separados por responsabilidade.
- ☐ Cenários aleatórios são determinísticos quando necessário.
- ☐ Campos de estado são privados e referências obrigatórias são finais.
- ☐ Não há listas internas mutáveis expostas.
- ☐ Não há objetos construídos apenas para serem substituídos.
- ☐ Não há imports inutilizados importantes ou métodos vazios.
- ☐ Não há regra importante implementada somente no App.
- ☐ O código usa uma língua e uma convenção de nomes consistentes.

Documentação e modelo

- ☐ README declara escopo e regras adotadas.
- ☐ README mostra como compilar, testar e executar.
- ☐ PUML possui delimitadores e renderiza.
- ☐ PUML contém Naipes e ValorCarta.
- ☐ Atributos, operações e visibilidades correspondem ao código.
- ☐ Relações e multiplicidades correspondem às referências reais.
- ☐ Extensões futuras estão separadas do MVP.

EVIDÊNCIA MÍNIMA

Cada marca precisa apontar para pelo menos uma destas provas: teste automatizado, execução manual reproduzível, código revisado ou diagrama renderizado.

Roteiro único de aceitação manual

Preencha observado e correção; naturais ficam na suíte automatizada, que usa baralho preparado.

Passo	Cenário	Resultado esperado
1	Menu	Mostra Blackjack e o) Sair; opções indisponíveis têm resposta.
2	Entrada não numérica	Mensagem útil e nova tentativa, sem exceção.
3	Nome completo	Nome com espaços é conservado; branco é rejeitado.
4	Iniciar rodada	Duas cartas por participante; 48 restantes.
5	Carta da banca	Existe na Mao, mas a segunda não aparece ao jogador.
6	Pedir	Mao +1 e Baralho -1; estado exibido permanece coerente.
7	Parar	Carta é revelada; banca executa a política S17.
8	Resultado	Vencedor e motivo coincidem com cartas e totais.
9	Nova rodada	Novo baralho e novas mãos; nome permanece; estado antigo não vaza.
10	Sair	Aplicação encerra claramente e sem exceção.

Registro curto de evidência

Passos	Observado	Correção necessária / evidência
1-3		
4-6		
7-8		
9-10		

COMO ACEITAR

Falha em qualquer passo reabre a fase responsável. Uma nova rodada deve provar novos objetos, não apenas uma tela limpa.

CAPÍTULO 08

08

Plano de estudo em doze sessões

Cada sessão produz uma evidência pequena. A nova rodada e a visibilidade da banca agora fazem parte do plano.

Sessão	Foco	Fase	Saída observável
1	Linha de base e referências	F0	Grafo correto; build limpo.
2	Mao persistente	F1	Vazia, adicionar e independência.
3	Participantes	F1	Nome válido e mãos próprias.
4	ValorCarta	F2	Nenhum rank impossível nasce.
5	Baralho completo	F3	52 combinações comprovadas.
6	Comprar e embaralhar	F3	Transferência, vazio e conjunto.
7	Blackjack limpo	F4	Sem dependências temporárias.
8	Distribuição e naturais	F4	2+2, 48, precedência e visibilidade.
9	Pontuação	F5	Figuras, ases e mão macia.
10	Estados, banca, resultado	F6	S17, Resultado e Motivo.
11	Interface e nova rodada	F7	Rodada completa com objetos novos.
12	Testes, UML e README	F8	Aceitação e build final.

Ritual de cada sessão

1. Preveja o comportamento atual e escreva o cenário que deve falhar.
2. Faça a menor alteração que satisfaça o contrato público.
3. Rode a classe de teste; depois, toda a suíte e mvn clean test no marco.
4. Explique a responsabilidade e registre qualquer decisão alterada no README/PUML.
5. Pare e divida se a sessão tocar mais de duas responsabilidades.

MARCO DA SESSÃO 11

Jogar novamente cria novo Baralho, novo Dealer e novas mãos. O teste deve usar `assertNotSame` nas referências de rodada e confirmar que o nome foi preservado.

Ficha reutilizável de implementação

Pergunta	Minha resposta
Problema observado	
Classe responsável	
Dados que essa classe já possui	
Contrato do método	
Pré-condição	
Pós-condição	
Teste que falha antes	
Menor implementação possível	
Armadilha que quero evitar	
Evidência de conclusão	

EXPLIQUE ANTES DE CODIFICAR

Se você não consegue completar 'esta operação pertence a X porque X já possui...', o lugar do método ainda não está claro.

CAPÍTULO 09

09

Pistas graduais

Consulte somente depois de tentar. Cada nível revela um pouco mais, sem fornecer classes completas.

1. Mao não conserva cartas

PISTA 1 - PERGUNTA

Que diferença existe entre uma variável declarada dentro e fora do construtor?

PISTA 2 - CONCEITO

O estado que precisa sobreviver deve ser campo de instância.

PISTA 3 - DIREÇÃO

Procure uma declaração `private final List<Carta>` no corpo da classe e inicialização no construtor.

2. Jogador perde a mão

PISTA 1 - PERGUNTA

Quem aponta para a Mao depois que o construtor termina?

PISTA 2 - CONCEITO

A variável local sai de escopo; o participante precisa conservar a referência.

PISTA 3 - DIREÇÃO

O campo e o parâmetro podem ter o mesmo tipo; use `this` para se referir ao campo da instância atual.

3. Carta aceita XPTO

PISTA 1 - PERGUNTA

Qual tipo permite apenas treze possibilidades?

PISTA 2 - CONCEITO

Um enum fecha o domínio e move determinados erros para o tempo de compilação.

PISTA 3 - DIREÇÃO

Faça Carta receber ValorCarta e faça Baralho percorrer os valores oficiais.

4. Comprar não reduz o baralho

PISTA 1 - PERGUNTA

O método devolve uma carta ou remove uma carta?

PISTA 2 - CONCEITO

Transferir exige remover da coleção de origem.

PISTA 3 - DIREÇÃO

Escolha o topo e use uma operação de remoção que também devolva o elemento.

5. Distribuição cria cartas novas

PISTA 1 - PERGUNTA

De onde deveria vir cada carta distribuída?

PISTA 2 - CONCEITO

Blackjack coordena a transferência da mesma referência do Baralho para Mao.

PISTA 3 - DIREÇÃO

A sequência conceitual é comprar e depois adicionar; não use `new Carta` na distribuição.

6. A + A soma 22

PISTA 1 - PERGUNTA

Quantos ases foram contados inicialmente como 11?

PISTA 2 - CONCEITO

Ajuste um ás por vez enquanto o total exceder 21.

PISTA 3 - DIREÇÃO

Cada ajuste subtrai exatamente 10; repita enquanto ainda houver ás flexível.

7. Teste de shuffle falha às vezes

PISTA 1 - PERGUNTA

A mesma ordem é matematicamente impossível?

PISTA 2 - CONCEITO

Aleatoriedade não promete mudança, apenas uma distribuição.

PISTA 3 - DIREÇÃO

Teste a preservação do conjunto ou forneça um Random previsível.

8. Nome completo quebra o menu

PISTA 1 - PERGUNTA

O que `next()` deixa no buffer?

PISTA 2 - CONCEITO

Ele lê um token, não a linha inteira.

PISTA 3 - DIREÇÃO

Leia a linha completa e converta opções textuais de maneira controlada.

CAPÍTULO 10

10

Glossário aplicado

Definições precisas ligadas ao código atual e às confusões que cada termo evita.

Termo	Definição aplicada ao Blackjack
MVP	Produto mínimo viável: menor versão jogável com regras explicitamente fechadas.
Linha de base (baseline)	Estado comprovado antes da mudança; permite detectar regressões.
Classe	Tipo que define estado e comportamento; Mao é classe, não a mão concreta.
Objeto	Instância criada em execução; duas chamadas a new Mao criam objetos distintos.
Instância	Objeto pertencente a uma classe.
Referência	Valor usado para alcançar um objeto; atribuir não clona o objeto.
Campo	Variável no corpo da classe, conservada pela instância.
Variável local	Nome temporário em método/construtor; sai do escopo ao terminar.
Parâmetro	Variável local que recebe uma cópia do valor fornecido na chamada.
Escopo	Região do código em que um nome pode ser usado.
Alcançabilidade	Existência de um caminho de referências até um objeto.
Estado	Dados que um objeto conserva entre chamadas.
Identidade	Ser aquele objeto específico; observável com assertSame.
Igualdade lógica	Equivalência por valor; usa equals e hashCode coerentes.
Construtor	Rotina que estabelece invariantes da nova instância.
Invariante	Condição verdadeira durante toda a vida válida do objeto.
Encapsulamento	Proteção de invariantes por API controlada; não é apenas private.
Visibilidade	Quem acessa um membro: private, package-private, protected ou public.
this	Referência da instância atual; separa campo de parâmetro homônimo.
final	Impede reatribuir a referência; não torna o objeto imutável.
static	Membro da classe, sem this; estado static é compartilhado.
Método de instância	Operação sobre um objeto específico, como baralho.comprar().
Coleção	Objeto que agrupa referências segundo um contrato.
List	Interface ordenada que permite duplicatas; a aceitação de null depende da implementação.
ArrayList	List indexada e mutável; aceita duplicatas e null, salvo regra da classe.
Generics	Restringem o tipo; não garantem unicidade, limite ou ausência de null.

Glossário aplicado - continuação

Termo	Definição aplicada ao Blackjack
Enum	Tipo com conjunto fechado de instâncias nomeadas; adequado a Naipes e ValorCarta.
Rank / ValorCarta	Identidade A, 2...10, J, Q, K; não é a pontuação final da Mão.
Valor-base	Contribuição inicial; o ás usa 11 e pode ser ajustado pela Mão.
Imutabilidade	Estado observável não muda depois da construção; Carta é candidata.
Valor derivado	Dado recalculado, como pontuação, mão macia e estouro.
Associação / dependência	Associação conserva referência; dependência é uso temporário.
Agregação / composição	Composição exige posse exclusiva; não é sinônimo de 'tem um'.
Herança	Relação 'é um'; não surge apenas para remover um campo repetido.
Responsabilidade	Motivo principal para uma classe existir e mudar.
Contrato / API pública	Pré-condições, efeitos, retorno e erros expostos sem vaziar detalhes.
Injeção de dependência	Receber colaboradores prontos, como Blackjack recebe Baralho.
Exceção	Sinaliza que uma operação não pode cumprir seu contrato.
Teste unitário	Verificação automatizada de uma regra por comportamento observável.
Arrange-Act-Assert	Preparar, executar uma ação e verificar seus efeitos.
Determinismo	Mesma preparação produz o mesmo resultado; essencial em testes.
Máquina de estados	Fases e transições permitidas durante a rodada.
Embaralhamento (shuffle)	Reordena sem mudar o multiconjunto; pode manter a mesma ordem.
Passagem por valor (pass-by-value)	Java copia valores; para objetos, copia a referência.
Aliasing / cópia defensiva	Referências podem compartilhar estado; copiar protege a coleção.
Mão macia	Ao menos um ás ainda conta como 11.
Blackjack natural	21 com duas cartas: ás e carta de valor-base 10.
Carta oculta	Carta na Mão da banca omitida só da apresentação até a revelação.
Resultado	Quem venceu: jogador, banca ou empate.
MotivoResultado	Por que terminou: natural, estouro, maior total ou igualdade.
Nova rodada	Novo conjunto de objetos; no MVP, apenas o nome é preservado.
S17	Regra em que a banca para em todo 17, inclusive soft 17.
Coletor de lixo (garbage collector)	Pode recuperar objetos inalcançáveis; não age em instante previsível.

ENCERRAMENTO

Sua próxima ação concreta

COMECE POR MAO

Transforme a coleção local em estado de instância, ofereça operações de adição e consulta de quantidade, e prove com testes que duas mãos são independentes. Não implemente pontuação ainda.

Sequência das próximas três entregas

1. Mao conserva cartas e seus testes passam.
2. Jogador e Dealer conservam mãos próprias e seus testes passam.
3. Carta recebe um tipo fechado de valor nominal e o Baralho continua com 52 combinações válidas.

Perguntas finais de professor

- Quem possui os dados necessários para executar esta regra?
- O valor precisa sobreviver ao método ou é apenas temporário?
- Que estado inválido o construtor deve impedir?
- Qual comportamento público prova que a mudança funcionou?
- O PUMML descreve o que existe hoje?
- Eu consigo explicar a decisão sem apontar apenas para a sintaxe?

Um bom projeto orientado a objetos não nasce de muitas classes. Ele nasce quando cada objeto conserva o estado pelo qual é responsável, protege suas invariantes e colabora por mensagens claras.

Fim do roteiro - agora a próxima linha é sua.